

VScode+AI verilog Program Solution

Introduction

Editor: VScode (Use the latest version, AI adaptation is better)

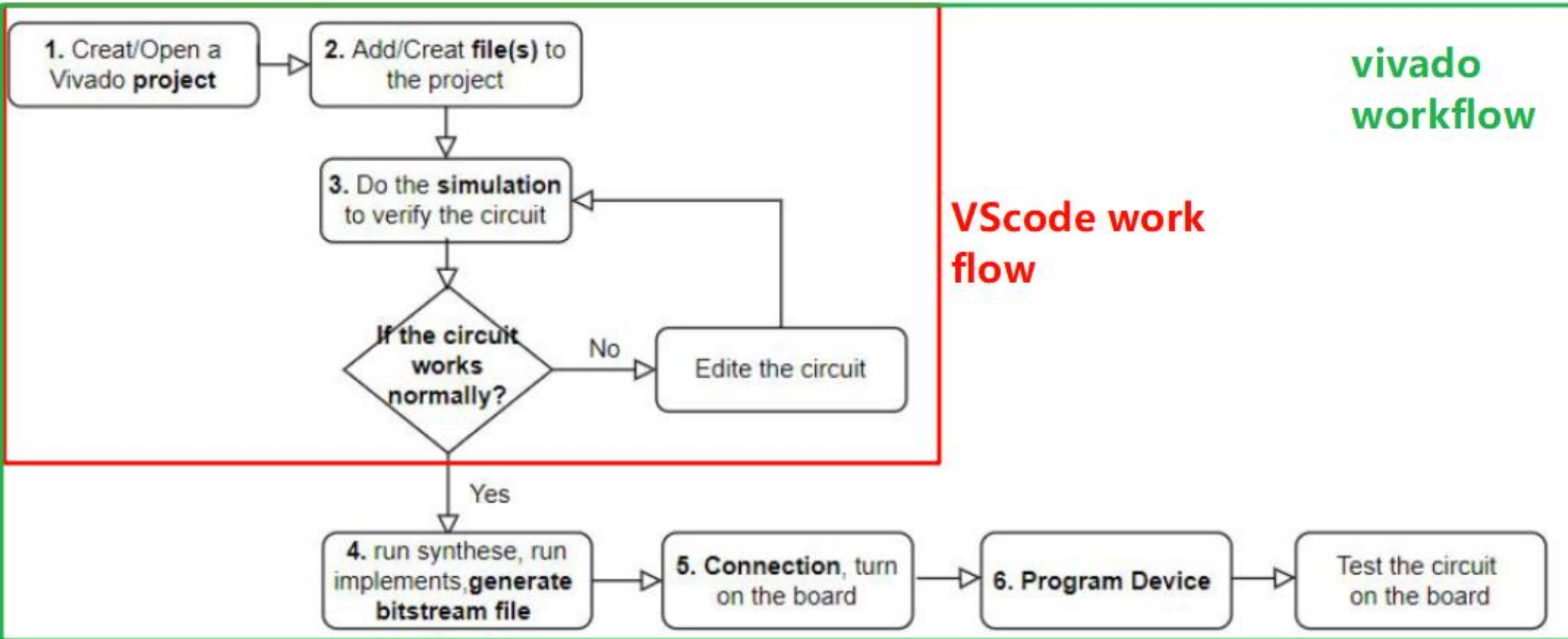
VScode plugins:

1. Github Copilot
2. Verilog-HDL/SystemVerilog/Bluespec SysyeVeril

Tools download:

1. Ctags: The purpose of ctags is to generate an index (tag) for symbols such as functions, classes, and variables in source code files, so as to quickly locate and jump.
2. iverilog: Iverilog is a Verilog HDL simulator that compiles and simulates Verilog code for digital circuit design and testing.

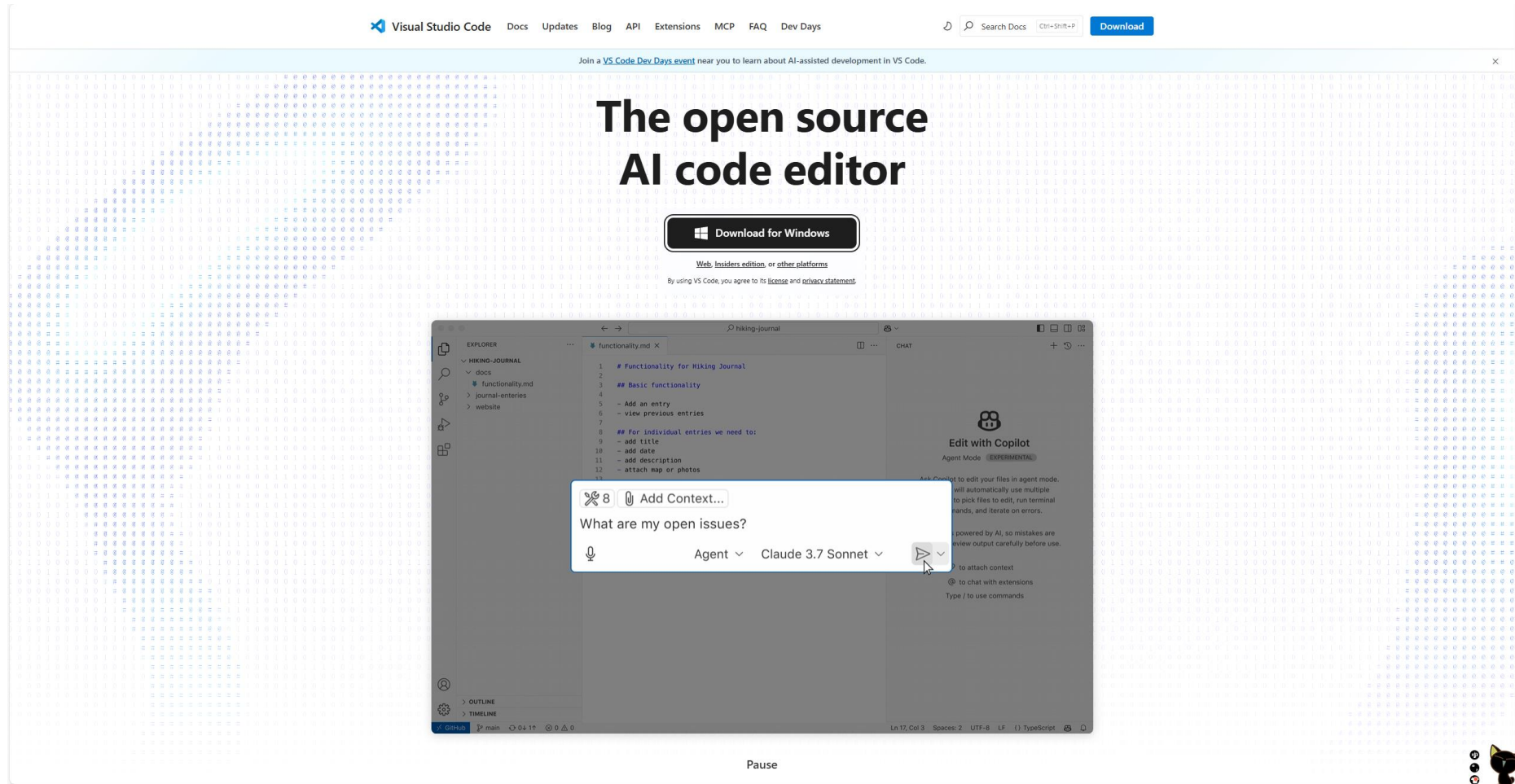
Solution Workflow



Installation Guide

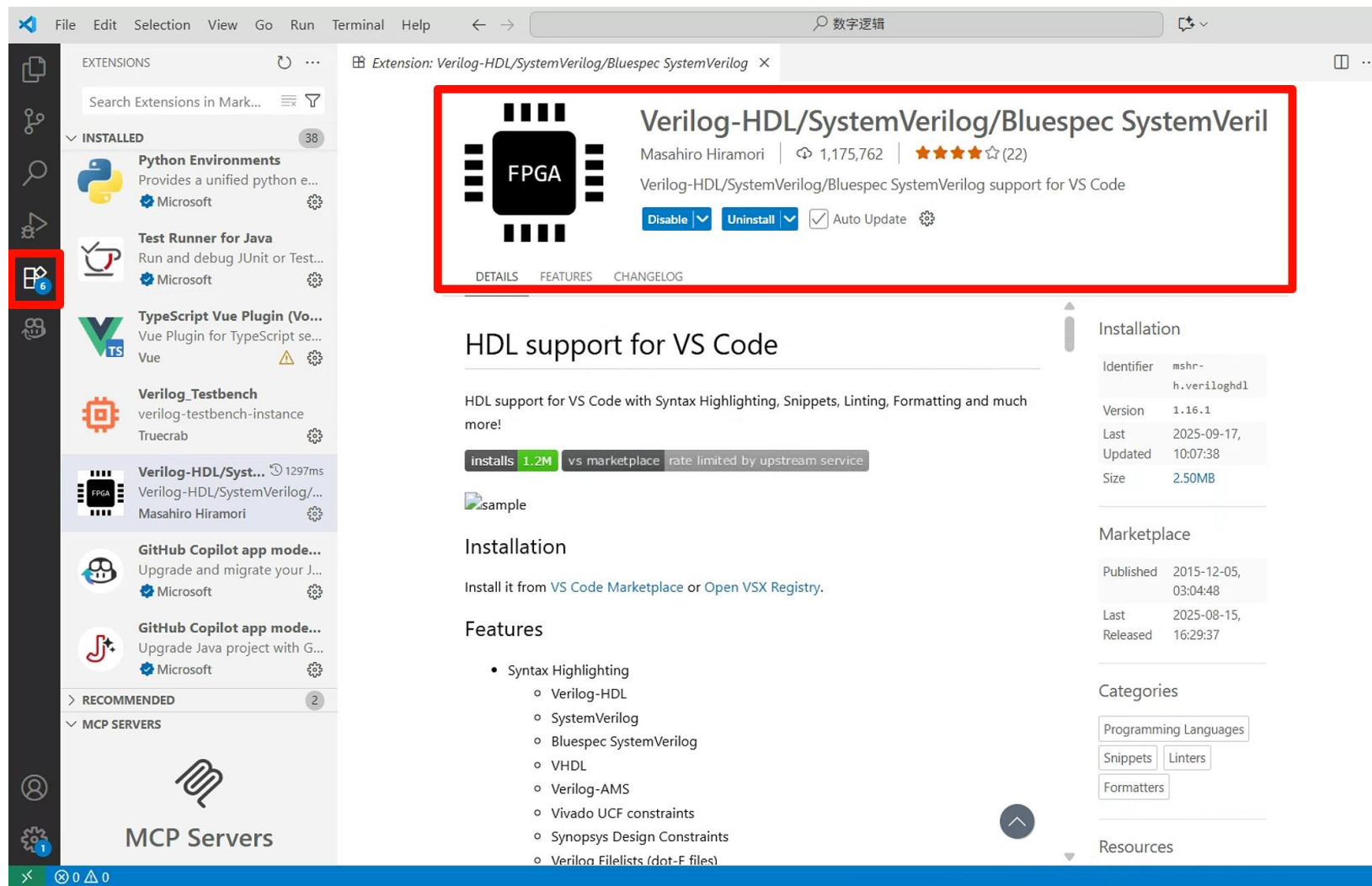
1. Download the VScode editor

[VScode download website](#)



Installation Guide

2. Follow the VScode Start guide to install Github Copilot
3. In order to use Copilot Pro, you need to [Github Student Developer Pack](#).
4. Go to the plugin market to download Verilog-HDL/SystemVerilog/Bluespec SystemVerilog



Installation Guide

5. Install ctags

[ctags download website](#)

6 hours ago
github-actions
2025-09-26/p...
9ea4b92
Compare

2025-09-26/p6.2.20250921.0-8-gd80d5de Pre-release

downloads@2025-09-26/p6.2.20250921.0-8-gd80d5de 33

Changes:

- Merge pull request #4305 from masatake/c#--nullable-parameters
- Merge pull request #4303 from masatake/Nftables

▼ Assets 8

ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-clang-x64...	sha256:1e7d5fe19b7cc3283...	2.91 MB	6 hours ago
ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-clang-x64...	sha256:744fd11a414b14ec4...	3 MB	6 hours ago
ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-x64.debu...	sha256:859c8a868ac4cba87...	3.74 MB	6 hours ago
ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-x64.zip	sha256:4ba609e27ca770db1...	3.02 MB	6 hours ago
ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-x86.debu...	sha256:8b7c495677a098014...	3.54 MB	6 hours ago
ctags-2025-09-26_p6.2.20250921.0-8-gd80d5de-x86.zip	sha256:0345bbe87350704f8...	3.05 MB	6 hours ago
Source code (zip)			6 hours ago
Source code (tar.gz)			6 hours ago

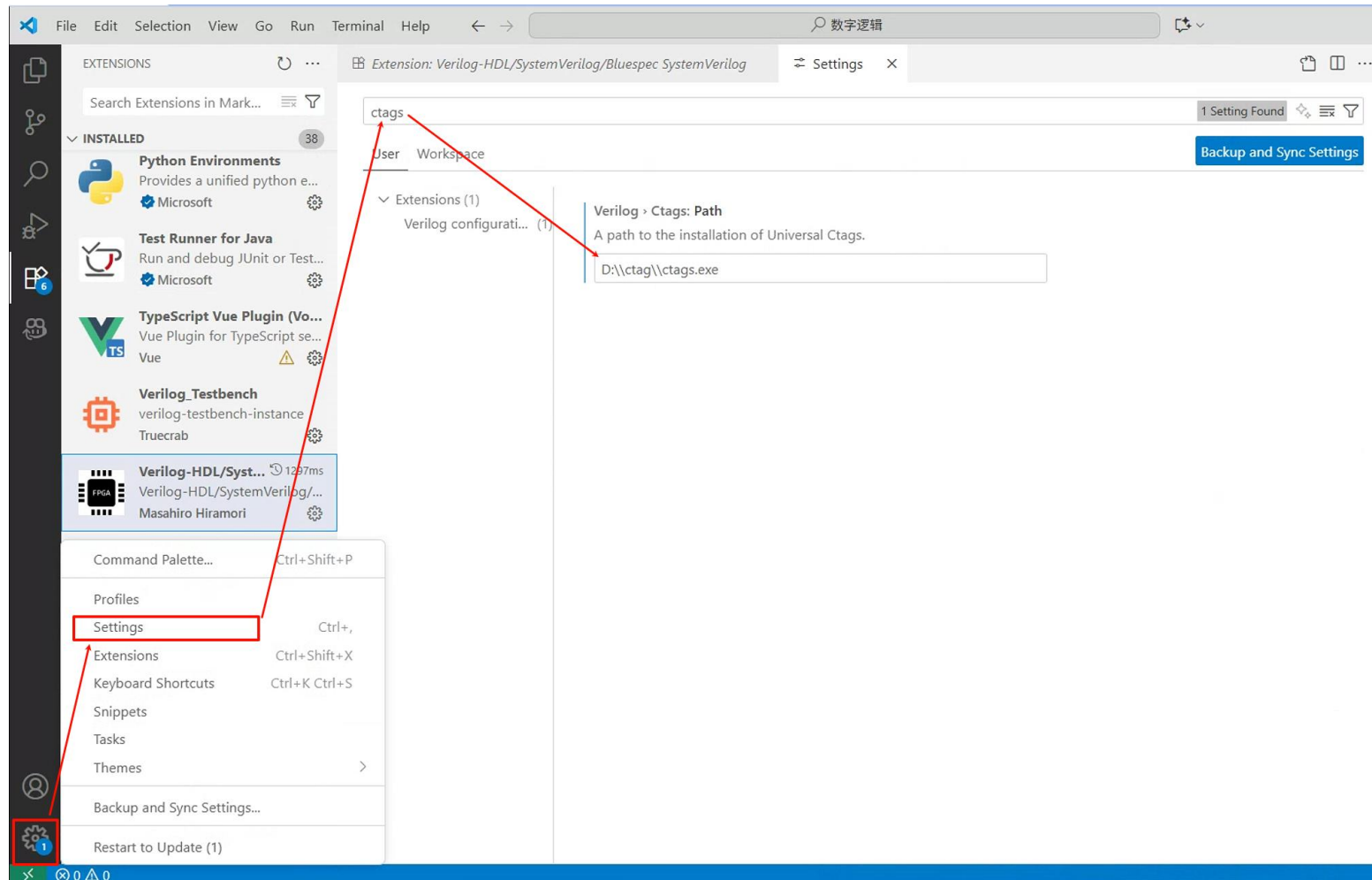
此电脑 > 新加卷 (D:) > ctag >

After downloading, unzip it to use.
No need to run the .exe file.

名称	修改日期	类型	大小
docs	2025/9/17 10:32	文件夹	
license	2025/9/17 10:32	文件夹	
man	2025/9/17 10:32	文件夹	
ctags.exe	2025/9/17 10:32	应用程序	5,01
README.md	2025/9/17 10:32	Markdown File	
readtags.exe	2025/9/17 10:32	应用程序	23

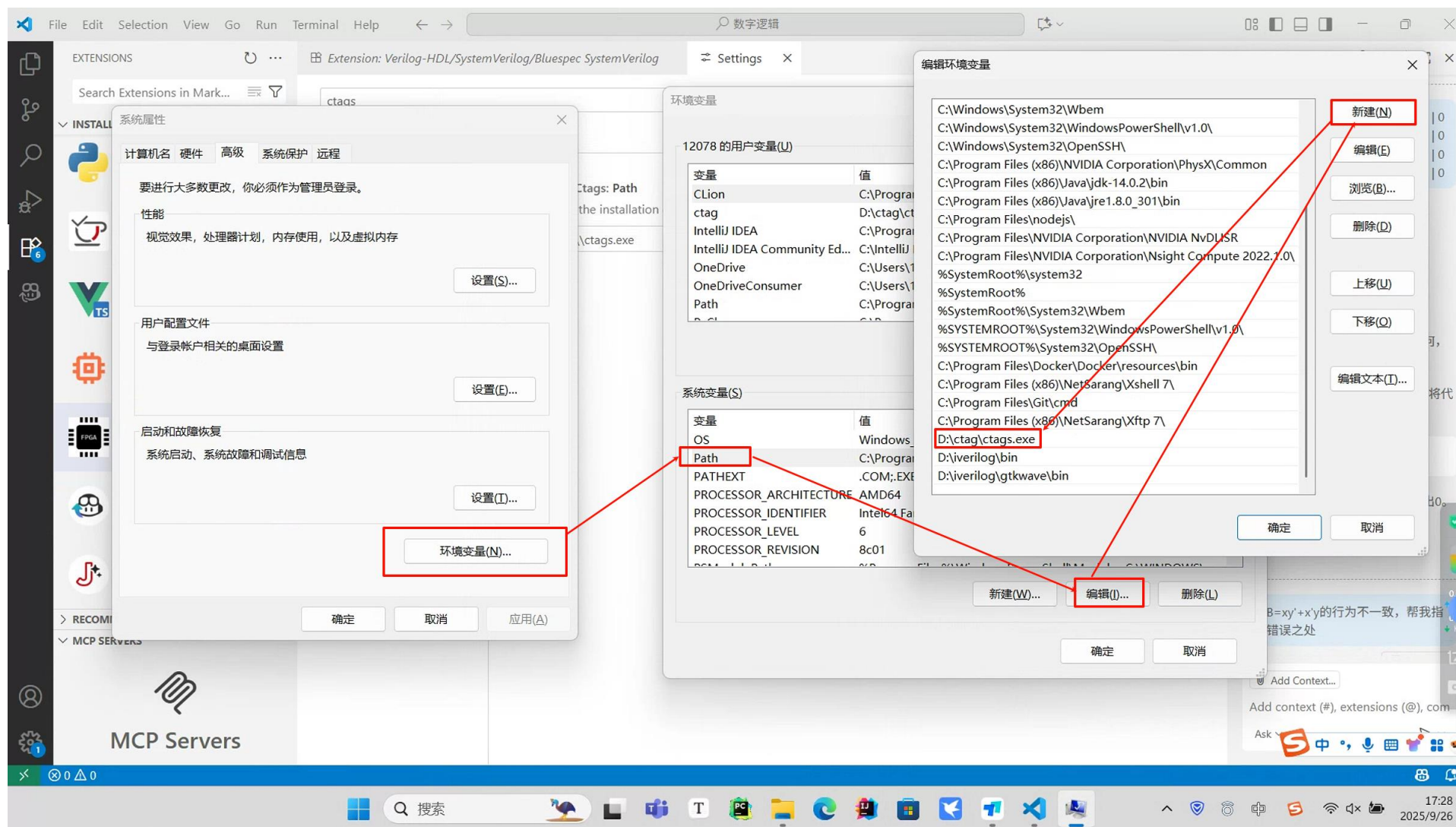
Installation Guide

5. Install ctags Configure ctags in VScode
vscode --> setting --> search ctags --> Enter the decompressed ctags program path



Installation Guide

5. Install ctags Configure system environment variables



6. Install iverilog

[iverilog download website](#)

After downloading, click the .exe file to run.



Icarus Verilog for Windows

Icarus Verilog is a free compiler implementation for the IEEE-1364 Verilog hardware description language. Icarus is maintained by Stephen Williams and it is released under the [GNU GPL license](#).

In this page you will find easy to install Icarus Verilog packages compiled with the [MinGW](#) toolchain for the Windows environment. [GTKWave for Win32](#) is also included in the latest releases. The installers have been created with Jordan Russell's [Inno Setup](#) free installer utility.

Download

You can find Icarus Verilog sources and binaries for most platforms at the [Icarus site FTP](#). The sources available here have been compressed with 7-zip.

- [iverilog-v12-20220611-x64_setup \[18.2MB\]](#)
- [iverilog-v11-20210204-x64_setup.exe \[44.1MB\]](#)
- [iverilog-v11-20201123-x64_setup.exe \[18.1MB\]](#)
- [iverilog-10.1.1-x64_setup.exe \[9.77MB\]](#)
- [iverilog-10.0-x86_setup.exe \[11.1MB\]](#)
- [iverilog-20130827_setup.exe \(development snapshot\) \[11.2MB\]](#)
- [iverilog-0.9.7_setup.exe \(latest stable release\) \[10.5MB\]](#)
- [iverilog-0.9.6_setup.exe \[10.4MB\]](#)
- [iverilog-0.8.6_setup.exe \(latest release 0.8 series\) \[1.29MB\]](#) [iverilog-0.8.6.7z \[800kB\]](#)
- [iverilog-0.7-20040706_setup.exe \[1.09MB\]](#) [iverilog-0.7-20040706.7z \[588kB\]](#)

Resources

Here are some pointers to interesting Verilog related resources.

- [Verilog Resources](#)
- [GTKWave Electronic Waveform Viewer](#)
- [GTKWave for Windows](#)
- [IVI](#), a graphical frontend for Icarus
- [Eclipse Verilog Editor](#)
- [Verilog syntax highlighting for UltraEdit](#).

Copyright

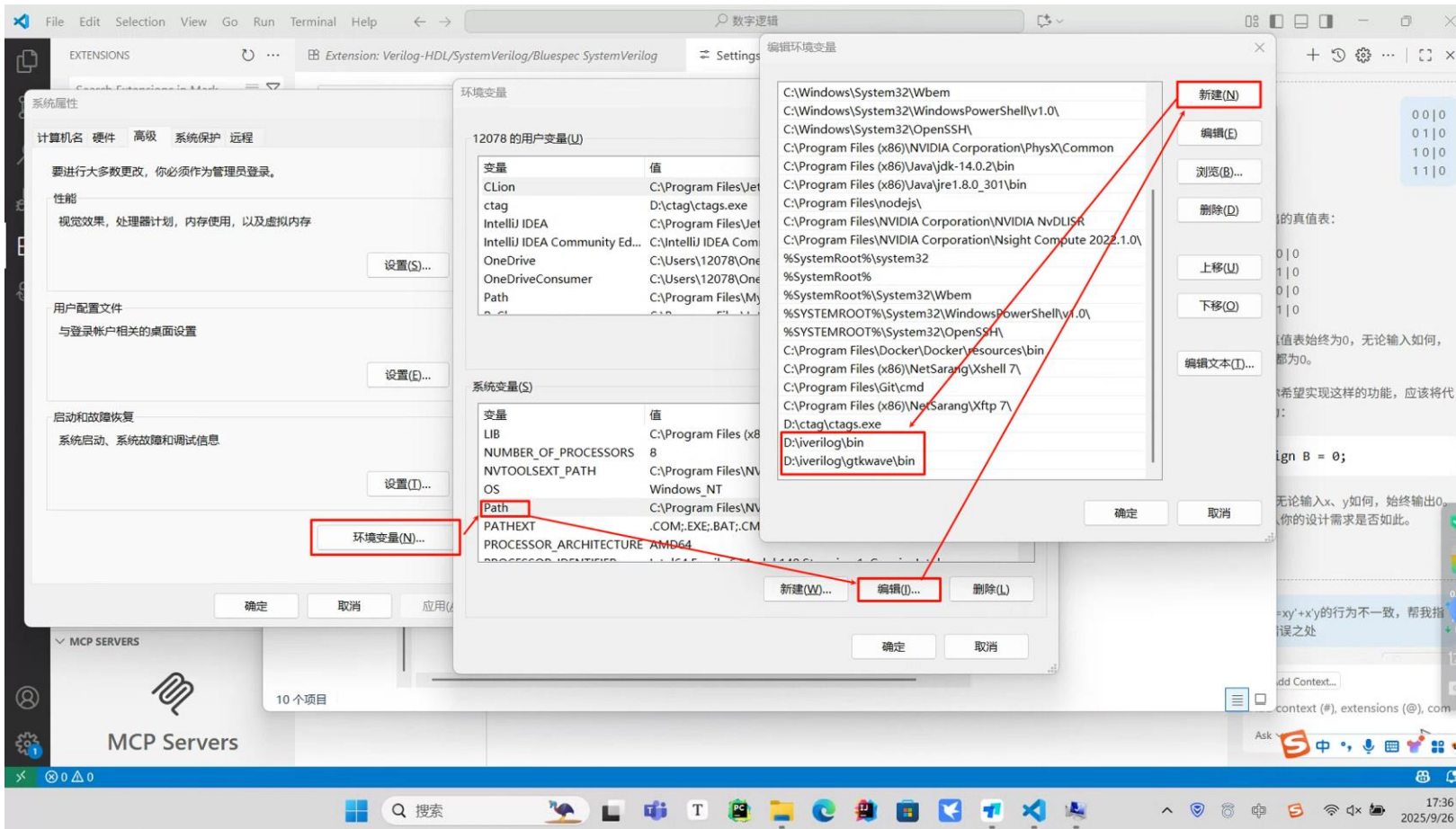
This program is free software; you can redistribute it and/or modify it under the terms of the [GNU General Public License](#) as published by the [Free Software Foundation](#); either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the [GNU General Public License](#) for more details.

Installation Guide

6. Install iverilog

Like ctags, you need to configure the system environment variables. Note that two variables need to be registered here.



After the registration is completed, restart VScode and open the terminal. Enter iverilog, and if the following image appears, the installation is successful.

```
(base) PS C:\Users\12078\Desktop\助教\数字逻辑> iverilog
D:\iverilog\bin\iverilog.exe: no source files.

Usage: iverilog [-EiSuvV] [-B base] [-c cmdfile] [-f cmdfile]
               [-g1995] [-g2001] [-g2005] [-g2005-sv] [-g2009] [-g2012] [-g<feature>]
               [-D macro[=defn]] [-I includedir] [-L moduledir]
               [-M [mode=]depfile] [-m module]
               [-N file] [-o filename] [-p flag=value]
               [-s topmodule] [-t target] [-T min[typ]max]
               [-W class] [-y dir] [-Y suf] [-l file] source_file(s)
```

See the man page for details.

7. Configure the verilog-HDL plugin

Search Settings for Linting, and select iverilog

The screenshot shows the VS Code interface with the 'Settings' window open. The search bar at the top of the settings window contains the text 'Linting', and it indicates '20 Settings Found'. The left sidebar shows the 'EXTENSIONS' view with the 'Verilog-HDL/SystemVerilog/Bluespec SystemVerilog' extension selected. The settings window is divided into 'User' and 'Workspace' tabs. Under the 'Workspace' tab, the 'Verilog > Linting > Iverilog: Run At File Location' setting is checked. Below this, the 'Verilog > Linting > Iverilog > System Verilog: Standard' setting is set to 'SystemVerilog2012'. The 'Verilog > Linting > Iverilog > Verilog HDL: Standard' setting is set to 'Verilog-2005'. The 'Verilog > Linting: Linter' setting is highlighted with a red box and is set to 'iverilog'. The 'Verilog > Linting > Modelsim: Arguments' setting is empty. The 'Verilog > Linting > Modelsim: Run At File Location' setting is unchecked.

EXTENSIONS

Search Extensions in Mark...

INSTALLED 38

Python Environments
Provides a unified python e...
Microsoft

Test Runner for Java
Run and debug JUnit or Test...
Microsoft

TypeScript Vue Plugin (Vo...
Vue Plugin for TypeScript se...
Vue

Verilog_Testbench
verilog-testbench-instance
Truecrab

Verilog-HDL/Syst... 1297ms
Verilog-HDL/SystemVerilog/...
Masahiro Hiramori

GitHub Copilot app mode...
Upgrade and migrate your J...
Microsoft

GitHub Copilot app mode...
Upgrade Java project with G...
Microsoft

RECOMMENDED 2

MCP SERVERS

Extension: Verilog-HDL/SystemVerilog/Bluespec SystemVerilog

Settings

Linting

20 Settings Found

Backup and Sync Settings

User Workspace

Extensions (20)
Verilog configurat... (20)

Verilog > Linting > Iverilog: Run At File Location

☐ If enabled, Icarus Verilog will be run at the file location for linting. Else it will be run at workspace folder. Disabled by default.

Verilog > Linting > Iverilog > System Verilog: Standard

Select the standard rule to be used when Icarus Verilog linting for SystemVerilog files.

SystemVerilog2012

Verilog > Linting > Iverilog > Verilog HDL: Standard

Select the standard rule to be used when Icarus Verilog linting for Verilog-HDL files.

Verilog-2005

Verilog > Linting: Linter

Select the verilog linter. Possible values are 'iverilog', 'verilator', 'modelsim', 'xvlog', 'slang' or 'none'.

iverilog

Verilog > Linting > Modelsim: Arguments

Add Modelsim arguments here. They will be added to Modelsim while linting.

Verilog > Linting > Modelsim: Run At File Location

☐ If enabled, Modelsim will be run at the file location for linting. Else it will be run at workspace folder. Disabled by default.

AI-assisted programming

Quick Inquiry: Use comments to write precise requirements.

Accurate Inquiry: Use the chat tool at the bottom right of VScode

The screenshot shows the VS Code editor with a Verilog file named `lab2.v`. A red box highlights a segment of code in the `tb_lab2` module, specifically the initialization and clock generation logic. A red arrow points from this box to the `Add Context...` button in the chat interface at the bottom right. The chat interface also shows a text input field and a button labeled `Ask`.

框选一个文件中的连续代码，可以指定文件的上下文范围，或者指定修改范围

add context可以引入指定文件上下文

```
4 module tb_lab2;
5
6 // lab2 Parameters
7 parameter PERIOD = 10;
8
9
10
11 // lab2 Inputs
12 reg [7:0] sw = 0;
13
14 // lab2 Outputs
15 wire [7:0] led;
16 reg clk = 0;
17 reg rst_n = 0;
18
19
20 initial
21 begin
22     forever #(PERIOD/2) clk = ~clk;
23 end
24
25 initial
26 begin
27     #(PERIOD*2) rst_n = 1;
28 end
29
30 lab2 u_lab2 (
31     .sw (sw [7:0]),
32     .led (led [7:0])
33 );
34
35
36 initial
37 begin
38     $finish;
39 end
```

```
// A = x'y + xy'
assign A = (x & ~y) | (~x & y);
```

After selecting the desired context range using add context and box selection, describe your issue in the chat box.

Note that box selection can only be performed within one file; other files can only be added as a whole text.

AI generates testbench

Select the design file, and in the chat tool, request to generate a simulation file. The statement requires GPT to generate a VCD file; otherwise, \$display code is used by default.

When providing a prompt, simple circuits may not require specific test scenarios. For complex circuits, it is recommended to actively provide test inputs or test scenarios.

When generating, GPT does not include design files; you need to add them manually for subsequent direct compilation and simulation file generation.

If you do not include an include statement, when using the iverilog directive, you need to manually add all design files, which will be troublesome during multi-file combinational circuit testing.

Add Context... lab2.v X

生成一个testbench文件，生成vcd文件记录波形图，不使用\$display函数输出

Ask GPT-4.1



```
`timescale 1ns/1ps
```

```
module lab2_tb;
```

```
reg x, y;  
wire B;
```

```
`include "lab2.v"
```

```
// 实例化待测试模块
```

```
lab2 uut (  
    .x(x),  
    .y(y),  
    .B(B)  
);
```

```
initial begin
```

```
    // 生成vcd波形文件
```

```
    $dumpfile("lab2_tb.vcd");
```

```
    $dumpvars(0, lab2_tb);
```

```
    // 输入激励
```

```
    x = 0; y = 0;
```

```
    #10;
```

```
    x = 0; y = 1;
```

```
    #10;
```

```
    x = 1; y = 0;
```

```
    #10;
```

```
    x = 1; y = 1;
```

```
    #10;
```

```
    $finish;
```

```
end
```

```
endmodule
```

Usage example

Iverilog Simulation \ GTK Waveform Visualization

Compile simulation file instruction:

1. Open terminal in vscode
2. Enter "iverilog -o {output file name} {simulation file name}".

```
iverilog -o .\lab2_tb.v.out .\lab2_tb.v
```

```
lab2.v lab2_tb.v x
lab2_tb.v > { } lab2_tb
1 `timescale 1ns/1ps
2 `include "lab2.v"
3
4 module lab2_tb;
```

3. Waveform Generation Code: Add the following code.

```
4 module lab2_tb;
10 lab2 uut (
11     .x(x),
12     .y(y),
13     .B(B)
14 );
15
16 initial begin
17     $display("x y | B");
18     x = 0; y = 0; #10; $display("%b %b | %b", x, y, B);
19     x = 0; y = 1; #10; $display("%b %b | %b", x, y, B);
20     x = 1; y = 0; #10; $display("%b %b | %b", x, y, B);
21     x = 1; y = 1; #10; $display("%b %b | %b", x, y, B);
22     $finish;
23 end
24
25 initial begin
26     $dumpfile("lab2_tb.vcd");
27     $dumpvars(0, lab2_tb);
28 end
29 endmodule
30
```

→ 波形图文件输出名

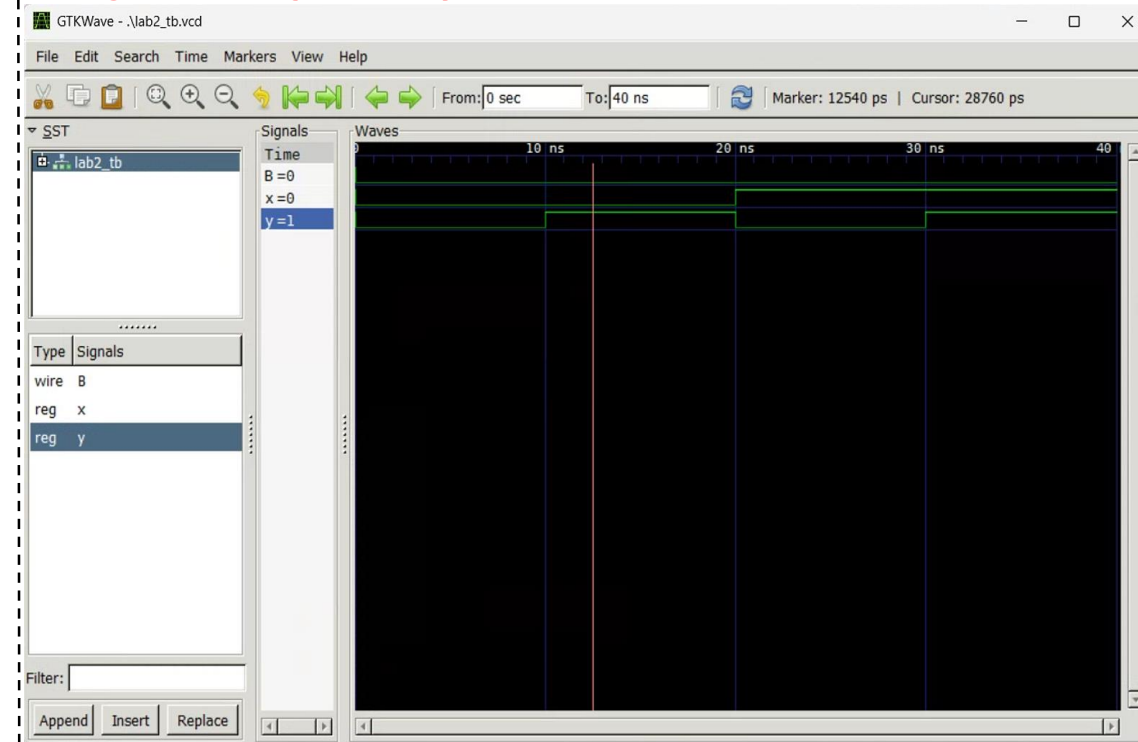
→ 注册波形图记录变量, 0表示展示lab2_tb中所有变量, 包括sub_module

Waveform generation and visualization instructions:

1. vvp {output file}

This output file is generated at step 2, and this step will generate a vcd file.

2. gtkwave {vcd file}



Basic Workflow

1. Compile the simulation file. Fix errors by locating the problematic code in the error message, selecting the file, and querying GPT with the details.
2. Check if the simulation waveform matches expectations. If not, report anomalies to GPT with: variable values, clock phase, and incorrect outputs.
3. If X appears in the simulation (indicating uncertainty), it signals a design flaw. Identify the abnormal variable and ask GPT to check its logic.

Tips

- If the simulation file test scenario involves multiple design files, it is necessary to check whether all context files have been added.
- Prepare a fixed error detection prompt (e.g., "Check for combinational logic loops, cross-module loops, multiple drivers, blocking/non-blocking assignment errors, and mixed usage in always blocks") to standardize post-design checks.

VScode+AI/verilog plugins VS Vivado

- Workflow
 - vscode: design + Simulation
 - vivado: Design + Simulation + Synthesis + Implementation + Generate bitstream
- Editor
 - vscode: It has a complete AI integrated development environment that can develop and debug more efficiently, with a simpler and more convenient UI.
 - vivado: The editor is quite primitive, the UI is complex, and reading error messages is difficult.
- File/Project Management
 - vscode: Directly manage the original folder, and later integrate Git development tools for efficient project version management.
 - vivado: Files need to be imported, and Git bash will be used for version management in subsequent project collaborations, which is more costly.
- Constraint File Generation
 - The constraint file is device-specific; directly generating one with AI risks pin mismatches. Using Vivado's constraint generation module is more efficient.

Thanks for listening!